1. Существующие проблемы

Проблемы воспроизводимости:

1) Отсутствие версионирования - невозможно откатиться к предыдущей версии модели или понять, какая версия сейчас в продакшене

2) Потеря контекста обучения - неизвестны гиперпараметры, версия кода, версия библиотек, окружения и конфигурация железа

3) Неясность данных - непонятно, на какой версии датасета обучена модель, какие фичи включены, за какое время

Проблемы обнаружения и сотрудничества:

1) Отсутствие поиска - невозможно найти модель по метрикам, автору или тегам (приходится перебирать файлы с комплексными составными названиями по папкам типа models/mlds_180/ model_v1_v0_without_rank_dataset_1/ :)))

2) Дублирование усилий - команды не знают о существующих наработках и обучают похожие модели заново

3) Сложность передачи артефактов, модель трудно передать коллеге или в CI/CD пайплайн, ситуации «А у меня всё работало»

Проблемы управления и безопасности:

1) Отсутствие жизненного цикла - не обозначены явно стадии разработки модели, не понятно, что сейчас в проде, а что еще в разработке. Старые модели могут просто так занимать место

2) Безконтрольный доступ - любой может удалить или перезаписать чужую модель

3) Разнородность форматов моделей и данных. Можно сохранить модель по разному, и потом есть вероятность, что коллега не воспроизведет эксперимент в точности. Нужен стандарт хранения моделей

2. Функциональные требования

1) Регистрация артефактов:

- Загрузка файла модели (бинарник/веса)
- Автоматическое сохранение метаданных: имя, версия, автор, дата, размер
- Привязка метрик качества (accuracy, f1, loss и т.д.).
- Привязка параметров обучения (гиперпараметры).
- Привязка контекста: Git Repo URL + Commit Hash, Python Environment (requirements.txt или docker image hash)
- Привязка данных: Ссылка на версию датасета + схема признаков (schema)

2) Управление жизненным циклом:

- Статусы модели (тест, прод, архив)
- Возможность пометить конкретную версию как прод

3) Поиск и обзор:

- Просмотр списка моделей с фильтрацией (по автору, метрике, дате, статусу)
- Просмотр деталей конкретной версии (скачивание, просмотр метаданных)
- Сравнение версий моделей (diff метрик)

4) Безопасность и доступ:

- Аутентификация пользователей
- Роли: админ (все права), пользователь (свои модели + чтение чужих)
- Видимость: приватная (только автор/админ), ограниченная (например, только доступ по команде), публичная
- Квоты - ограничение на размер файла и количество загрузок в сутки (защита от злоупотреблений).

2. Нефункциональные требования

Производительность:

- Latency API: Ответы API (список моделей, метаданные) < 200 мс
- Поддержка одновременной загрузки/скачивания минимум 50-70 пользователей без деградации
- Время регистрации модели: < 5 сек (не считая времени передачи файла)

Надежность и доступность:

- Система должна быть доступна 99% времени (для MVP)
- Гарантия целостности данных - файл, скачанный из реджистри, должен совпадать с загруженным
- Резервное копирование - метаданные (БД) должны бэкапиться

Масштабируемость:

- Хранилище файлов должно быть отделено от приложения (чтобы можно было подключить S3/MinIO и не забивать диск сервера)
- Архитектура должна позволять горизонтальное масштабирование API (через Docker/kubernetes в будущем)

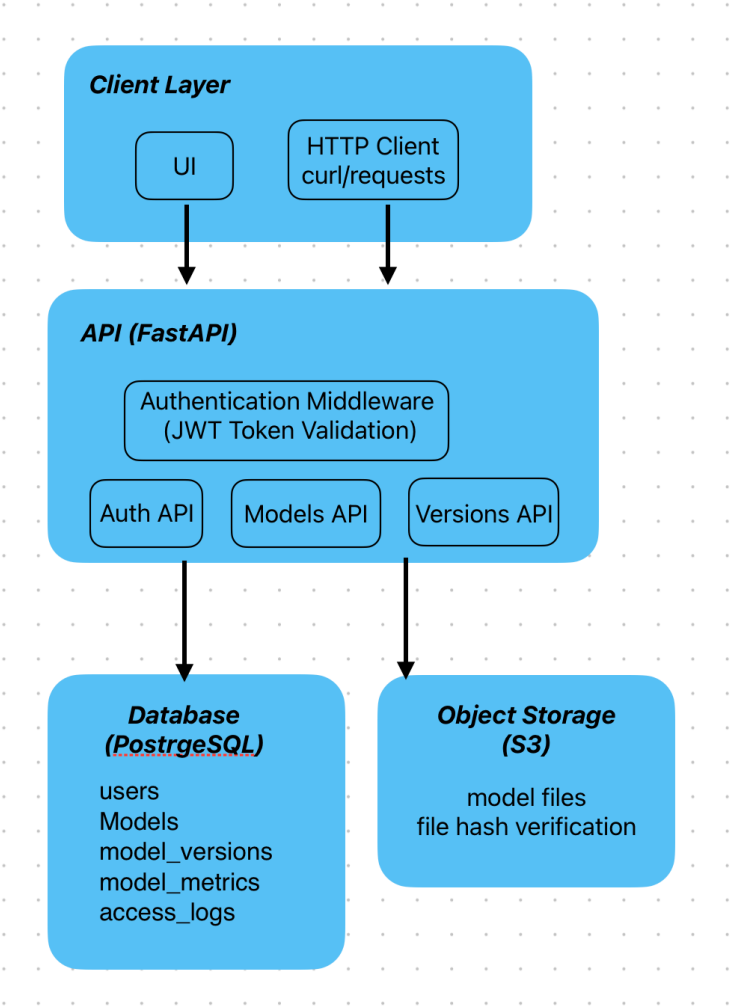
Безопасность:

- Все соединения по HTTPS
- Пароли хранятся в хешированном виде
- Валидация загружаемых файлов (проверка magic numbers, чтобы не загрузили .exe под видом .pkl)

Удобство использования:

- Наличие CLI-клиента для ML-инженеров (удобнее грузить из скрипта, чем через UI)
- Наличие простого Web UI для просмотра
- API должно быть RESTful

3. Архитектура, API, схема БД



Выбранный стек технологий

API Framework	FastAPI	Асинхронный, автоматическая документация, валидация данных через Pydantic
Database	PostgreSQL	Реляционная модель подходит для связей модель-версия-метрики
Object Storage	S3	Файлы моделей могут быть большими (ГБ). S3 позволяет масштабироваться
Authentication	JWT (JSON Web Tokens)	легко масштабировать, стандарт
Password Hashing	bcrypt / passlib	Безопасное хранение паролей

Схемы API

Аутентификация

POST	/api/v1/auth/register	Регистрация нового пользователя
POST	/api/v1/auth/login	Аутентификация, получение JWT токена
POST	/api/v1/auth/refresh	Обновление JWT токена
GET	/api/v1/auth/me	Получение информации о текущем пользователе

Модели

GET	/api/v1/models	Список моделей (с фильтрами и пагинацией)
POST	/api/v1/models	Создание новой модели (метаданные)
GET	/api/v1/models/{model_id}	Получение деталей модели
PUT	/api/v1/models/{model_id}	Обновление модели
DELETE	/api/v1/models/{model_id}	Удаление модели

Версии моделей

GET	/api/v1/models/{model_id}/versions	Список версий модели
POST	/api/v1/models/{model_id}/versions	Создание новой версии модели (загрузка артефакта)
GET	/api/v1/versions/{version_id}	Получение деталей версии
PUT	/api/v1/versions/{version_id}	Обновление версии
DELETE	/api/v1/versions/{version_id}	Удаление версии
GET	/api/v1/versions/{version_id}/download	Скачивание файла модели
PUT	/api/v1/versions/{version_id}/status	Изменение статуса версии (staging/production/archived)

Метрики

GET	/api/v1/versions/{version_id}/metrics	Получение метрик версии
POST	/api/v1/versions/{version_id}/metrics	Добавление/обновление метрик версии

Таблицы PostgeSQL

```
-- Users table
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) DEFAULT 'user' CHECK (role IN ('admin', 'user')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Models table
CREATE TABLE models (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(100) NOT NULL,
    description TEXT,
    owner_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    visibility VARCHAR(20) DEFAULT 'private' CHECK (visibility IN ('private', 'limited', 'public')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Model versions table
CREATE TABLE model_versions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_id UUID NOT NULL REFERENCES models(id) ON DELETE CASCADE,
    version VARCHAR(50) NOT NULL,
    status VARCHAR(20) DEFAULT 'staging' CHECK (status IN ('staging', 'production', 'archived')),
    description TEXT,
    file_path VARCHAR(500) NOT NULL,
    file_hash VARCHAR(64) NOT NULL,
    file_size BIGINT NOT NULL,
    data_info JSONB DEFAULT '{}',
    code_info JSONB DEFAULT '{}',
    environment JSONB DEFAULT '{}',
    hyperparameters JSONB DEFAULT '{}',
    metrics JSONB DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (model_id, version)
```

);